

Design Rationale: LEGO House Database Solution

1. Introduction

Project Objective

The objective of this project is to design a relational database capable of modelling the construction of LEGO houses.

The solution must:

- Support a reusable catalogue of LEGO parts and colours
- Support multiple house designs
- Represent house structures and assemblies
- Manage Bills of Materials (BOM)
- Enforce relevant business rules and data integrity
- Remain simple enough for both technical and non-technical users to understand

Design Approach

The solution follows a modular design approach by separating:

- Catalogue data
- House design data
- Bill of Materials data

This separation promotes reusability, scalability and maintainability.

2. Design Principles

Simplicity

The database has been designed using a small number of entities with clearly defined responsibilities.

This approach reduces complexity and makes the system easier to understand, implement and maintain.

Reusability

Parts and colours are stored independently from house designs.

This allows the same catalogue items to be reused across multiple houses without duplication.

Data Integrity

Primary keys, foreign keys and database constraints are used to maintain referential integrity and prevent invalid data from being entered.

Scalability

The structure allows new houses, rooms, parts and colours to be added without requiring changes to the database design.

3. Entity Design Decisions

3.1 House

Purpose

Stores individual house designs.

Design Rationale

The House entity acts as the highest level of the model and allows multiple house designs to exist within the same system.

Key Attributes

- HouseID
- HouseName

3.2 HouseComponent

Purpose

Stores physical components of a house.

Design Rationale

House components represent physical structures that can be built using LEGO parts.

These include:

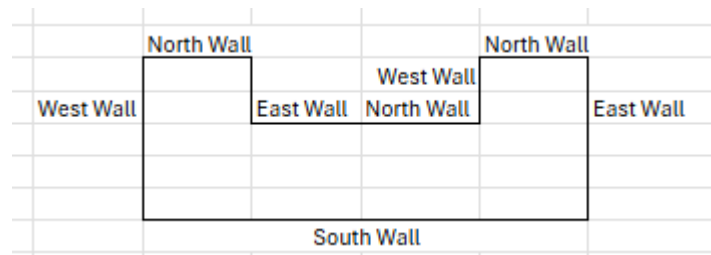
- Walls
- Floors
- Roofs
- Doors
- Windows

Each component can have its own Bill of Materials.

An alternative design was considered where house components (e.g. North Wall, South Wall) were stored independently and linked to houses through another table. This was rejected because it introduced additional complexity that was not required for the project scope.

A 'ComponentType' attribute was added to support the classification of other components and simplify the querying process. For example, all wall components can be retrieved without needing to search for specific individual component names such as 'North Wall'.

The design does not restrict a house to a single wall component because a house may contain multiple walls. Each wall has therefore been recorded as an individual component. Illustration below:



Doors and windows are represented as both house components and physical parts to reflect their dual role as structural elements of the house and as individual items required for construction.

Component IDs were standardised using a consistent identifier from the initial draft to improve readability and maintain consistency across the database.

Key Attributes

- ComponentID
- HouseID
- ComponentType
- ComponentName

3.3 Room

Purpose

Stores rooms within a house.

Design Rationale

Rooms have not been modelled as separate entities because rooms are not physical assemblies that consume LEGO parts. Instead, physical components such as walls, floors, roofs, doors and windows are represented within the data model and associated with the Bill of Materials. Room requirements are satisfied through the house design specification rather than through part construction and quantities of rooms can be proven through queries.

3.4 Part

Purpose

Stores parts available for construction.

Design Rationale

Parts are stored independently from houses so they can be reused across multiple house designs, thus supporting possible future expansion. This allows for multiple part variations to exist within a broader category whilst remaining consistent – for example, several brick types could be introduced later without it altering the data model (such as 'sandstone', 'fire' etc.)

Key Attributes

- PartID
- PartName
- Category

3.5 Colour**Purpose**

Stores available colours.

Design Rationale

Colours are stored separately to reduce duplication and simplify maintenance.

Key Attributes

- ColourID
- ColourName

3.6 PartColour**Purpose**

Represents valid combinations of parts and colours.

Design Rationale

The PartColour entity acts as a junction table, resolving the many-to-many relationship between Parts and Colours and ensures only valid combinations can be used. This reduces data duplication by avoiding the need to store colour information within the Parts entity. It also provides flexibility for a single part to be available in multiple colours. This improves the possibility of scalability as new parts could be added without changing the database structure.

The design was refined to minimise duplicate data by storing part and colour information separately and then linking them through the PartColours table.

Key Attributes

- PartColourID
- PartID
- ColourID

3.7 BillOfMaterials**Purpose**

Stores the quantity of coloured parts required to construct each house component.

Design Rationale

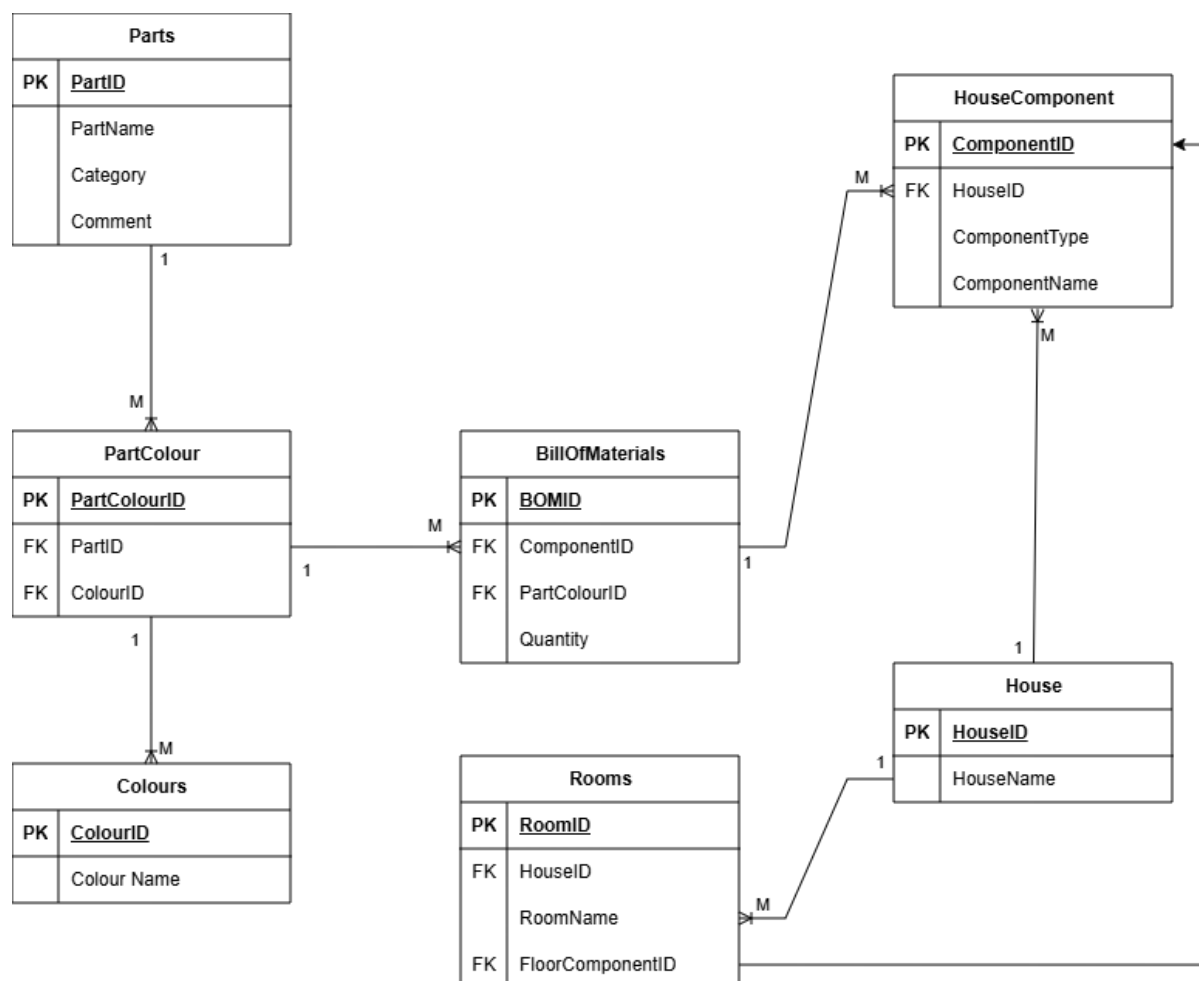
The BOM links physical house components with the parts needed to build them.

Quantities are used instead of repeating records to improve efficiency and satisfy the challenge requirements.

A validation constraint was applied to the Quantity attribute to ensure values are greater than zero. This helps maintain data integrity by preventing invalid Bill of Materials records from being entered, such as components with zero or negative quantities.

Key Attributes

- BOMID
- ComponentID
- PartColourID
- Quantity



4. Key Relationships

The database contains the following relationships:

Relationship	Cardinality
House → HouseComponent	One to Many
House → Room	One to Many
Floor → HouseComponent*	One to Many
PartColour → Part	One to Many
Colour → PartColour	One to Many
HouseComponent → BillOfMaterials	One to Many
PartColour → BillOfMaterials	One to Many

* FloorComponentID is a foreign key to HouseComponent.ComponentID and is intended to reference components where ComponentType = 'Floor'.

5. Business Rule Implementation

Business Rule	Implementation	Evidence
A house can contain multiple components	House -> HouseComponent relationship	H004 linked to C001-C011
Each house must contain at least two rooms	Room entity and validation query	Room count query returns ≥ 2 rooms per house
A house can contain multiple rooms	House -> Room relationship	Evidenced through COUNT(*) GROUP BY HouseID queries
Rooms must belong to a floor	FloorComponentID foreign key	Component C006
Parts may exist in multiple colours	PartColour entity	BK001 linked to R001, R002 in PartColours
Quantities must be positive	CHECK constraint	Constraint definition and BOM quantities all greater than 0

Duplicate part-colour combinations are not permitted	UNIQUE constraint	UNIQUE constraint on PartID and ColourID combination
Invalid references are not allowed	Foreign keys	Foreign Key definitions throughout SQL
Components are reusable within house designs	HouseComponent structure	Components are stored individually and referenced through the BOM
Walls must be constructed from brick parts	Validated through BOM quantities and house build queries	Components C001-C004 and C011 use BK001(Brick) in BOM
Walls must contain the required number of bricks (e.g. 40 or 50)	Validated through BOM quantities	BM001-BM004 and BM0012 specify wall brick quantities
Only brick parts may be used for wall construction	Part category validation and BOM review	Wall BOM reference Part BK001 (Brick) only
A roof must contain both tile and rafter parts	Enforced through roof BOM composition and validated through queries	BM005(Roof Tile) and BM006(Roof Rafter) link to C005(Roof)
A roof consists of 30 roof tiles and 8 rafters	Represented in sample BOM data (Roof Tile x30 and Roof Rafter x8)	Roof component contains Roof Tile and Roof Rafter quantities as defined in sample data
Only roof tile and rafter parts may be used within roof assemblies	Validated through part category and BOM review	Roof BOM reference TR001(Tile) and RR001(Rafter) only
Bill of Materials should show quantities such as 'Brick x5' rather than multiple identical rows	Single BOM record per component/part combination with a quantity value	BM001 stores Brick quantity as 40 rather than 40 separate records

Basic data integrity rules are enforced using primary keys, foreign keys, unique constraints and check constraints. More complex construction rules, such as specific part quantities for walls and roofs, are validated through business rules and reporting queries rather than database constraints alone. Required relationships are enforced through the use of NOT

NULL and foreign key constraints. For example, a PartColour record cannot exist unless it is linked to a valid Part and a valid Colour. This prevents the creation of orphaned records.

6. Assumptions

The following assumptions were made during development:

- A house may contain more than the minimum required number of rooms.
- A house may contain more than four walls if required.
- Doors and windows are treated as house components.
- Rooms represent logical spaces and are not directly built from parts.
- Components are considered physical assemblies.
- The part catalogue is shared across all houses.

7. Future Enhancements

Potential future improvements include:

- Support for multiple floors
- Support for different roof designs
- Inventory management
- Cost calculations
- House versioning and design history

Conclusion

The proposed data model provides a simple, scalable and maintainable solution for modelling LEGO house construction. By separating catalogue data from house design data and implementing key business rules through relationships and constraints, the design supports reusability, data integrity and future expansion.